BYOP Project Report: Smart-Soil Crop Recommender

Course: Fundamentals in AI and ML

1. The Problem and Why It Matters While studying environmental systems recently, I was struck by how much the specific balance of the nitrogen and phosphorus cycles dictates what can actually survive in a given ecosystem. It made me think about local agriculture. A lot of farming decisions are still based on historical habits or guesswork rather than hard soil data. If a farmer plants a crop that needs high nitrogen in soil that's depleted, they end up over-fertilizing, which causes chemical runoff and hurts the local water supply. I wanted to build a machine learning tool that takes the guesswork out of this process by recommending the absolute best crop for a specific soil's NPK (Nitrogen, Phosphorus, Potassium) levels and climate.

2. My Approach to Solving It I decided to frame this as a multi-class classification problem using Supervised Machine Learning. My inputs were the environmental features (NPK levels, temperature, humidity), and the target output was the crop name.

Interestingly, one of my first hurdles was that a lot of the standard datasets online were either too massive and messy, or they didn't map well to the specific thresholds I wanted to test. To solve this, I actually wrote a Python script to generate my own synthetic dataset. I programmed it so that crops like "Rice" specifically required high humidity and nitrogen, while "Chickpeas" tolerated drier conditions, mimicking real biological rules.

3. Key Decisions Made

- **The Algorithm:** At first, I thought about just using a basic Logistic Regression model. But crop survival isn't really linear—a plant might need *exactly* moderate phosphorus, and too much or too little will both ruin the yield. Because of these strict thresholds, I went with a **Random Forest Classifier**. It handles non-linear rules perfectly by building a bunch of decision trees, which felt a lot closer to how environmental conditions actually work.

- **Dockerizing the Project:** I decided to include a Dockerfile in my repository. I've realized that a good model isn't very useful if someone else struggles to run the code due to version conflicts, so I wanted to make sure the environment was containerized and easily reproducible.

4. Challenges Faced The biggest challenge wasn't actually training the model—it was figuring out how to explain *why* it was making its choices. For a while, the Random Forest just felt like a black box. It took me a bit of trial and error with scikit-learn and seaborn to extract the feature_importances_ data and get it to plot correctly on a graph. It was a bit frustrating formatting the chart, but getting that visual output was a breakthrough moment for the project.

5. What I Learned This project was a great reality check on how ML actually works in practice. I learned that writing the algorithm is surprisingly the fastest part of the job. The real work is in engineering the data, evaluating the metrics (like understanding why F1-score is sometimes better than pure accuracy), and interpreting the results. When my feature importance graph finally generated and showed that Humidity and Nitrogen were driving 60% of the model's decisions, it was really rewarding. It proved the AI was actually picking up on the real-world agricultural science I programmed into the data, rather than just guessing.